CGE-P LAB NOTES

CGE-P lab notes, lab by lab

Eleven labs and a capstone brief. Every lab produces an artifact the capstone consumes.

For reading. Code lines wrap to fit the page; copy code from docs/README.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/README.md` in your clone, not from the PDF.

Where these live. Upstream, the guides are `guides/*.md` in `GRCEngClub/cgep-labs` and the companion code is `reference/lab-X-Y/`. These drafts sit in `docs/` so you can build the reference workspaces from them before deciding what to upstream. Every code block here is written to pass that repo's CI: `terraform fmt -check -recursive`, `terraform validate -backend=false`, `opa test`, and `trestle validate`.

These notes differ from the official labs in one governing way: **a lab may only cite a control it actually implements.** Where a citation and the code did not line up, the code was written or the claim was moved to the lab that earns it. Where a mapping is arguable, the lab says so and defends it, because teaching students that control mappings are arguments rather than lookups is the single most transferable thing in this course.

Start here

The Plain English Guide is the companion to everything below. The labs tell you what to type; that guide explains what it means, where every magic string and constant came from, and which choices are actually yours. Read it first, or keep it open alongside the labs. It is not a lab and creates nothing.

Reading order

#	Lab	Cloud	Produces
0.0	Plain English Guide	none	Concepts, decoder ring, decision tables. **New in v2.** Not a lab.
0.1	Prerequisites & Credentials	both	Sandbox account, short-lived credentials, toolchain, **budget alarms**. **New in v2.**
2.2	Remote State Backend	AWS	S3 + DynamoDB state backend. **New in v2.**
2.3	First Compliant Resource	AWS	`compliant-s3` primitive with CMK, TLS enforcement, lifecycle.
2.4	Modules for Compliance	GCP	Reusable module discipline, CMEK, consumer pattern.
2.5	IaC as Compliance Evidence	AWS	Object Lock evidence vault + `capture-evidence.sh`.
3.3	Writing Rego	GCP	Policy library with metadata and test fixtures.
3.4	Conftest + Terraform	AWS	AWS policy variants + `policy-gate.sh`.
4.3	GRC Evidence Pipeline	AWS	`.github/workflows/grc-gate.yml`.

#	Lab	Cloud	Produces
4.4	Chain of Custody	AWS	Cosign signing wired into the vault.
5.2	AWS Security Services	AWS	CloudTrail, Config, Security Hub, Athena review.
5.4	GCP Security Services	GCP	Org Policy, Workload Identity Federation, Data Access logs.
6.1	Introduction to OSCAL	none	Component definition + profile, validated.
7.1	Capstone Brief	AWS	The graded deliverable.

Read the starter's gaps early

The capstone hands you a real, deliberately broken application: `GRCEngClub/cgep-app-starter`. Its `GAPS.md` names eight compliance gaps you are asked to close, and the whole point is that you are governing code you did not write, which is harder and more realistic than governing your own.

Read that file before Lab 2.3, and skim the starter's Terraform with it. Ten minutes, no setup, nothing to deploy. Every lab after it changes character: you stop learning SC-8 as an idea and start recognizing the bucket that is missing it. Labs 2.3, 3.4 and 5.2 carry callouts naming which gaps that lab's work addresses.

You do not need to fix all eight. Five, closed with depth and traceable through your OSCAL, is what passes.

One repository, built up

The labs are cumulative. Each one produces something the next consumes, and by Lab 6.1 you are holding the whole structure the capstone is graded on. The full tree, with which lab creates each part, is in Lab 0.1 Step 12d.

The short version of the chain:

Lab	Produces	Consumed by
2.2	the state backend	every workspace after it, and Lab 4.3's pipeline
2.3	the `compliant-s3` pattern	2.5's vault, 5.2's buckets, the capstone
2.4	the GCP module and its attestation	6.1's OSCAL component
2.5	the evidence vault and capture script	4.3, 4.4, 5.2's data events, 6.1's links
3.3	the policy library and catalog	3.4's gate, 4.3's CI, 6.1's requirements
3.4	`policy-gate.sh`	4.3, which shells out to it rather than reimplementing
4.3	the pipeline	4.4, which adds signing to it

Lab	Produces	Consumed by
4.4	signing and verification	6.1's evidence links, the capstone's chain of custody
5.2 / 5.4	the cloud baselines	the capstone's Layer 1
6.1	the OSCAL component and profile	the capstone's write-up

Nothing is discarded between labs. If a step tells you to delete something, it is cleanup of a cloud resource that costs money, never of an artifact you built.

The shared baseline

Every S3 bucket built anywhere in this curriculum enforces the same floor. Labs do not re-argue it; they cite it. If you change something here, it changes everywhere, which is the point.

Property	Resource	Control	Why it is in the floor
Customer-managed KMS key, rotation on	`aws_kms_key`	SC-12, SC-13	Key custody and rotation are yours, not the platform's.
SSE-KMS with bucket keys	`aws_s3_bucket_server_side_encryption_configuration`	SC-28, SC-28(1)	Bucket keys cut KMS request cost by up to 99%, and are **required** when the bucket receives S3 server access logs.
Versioning	`aws_s3_bucket_versioning`	CP-9, SI-7	Prior object states survive deletion and overwrite.
All four public-access-block flags	`aws_s3_bucket_public_access_block`	AC-3	Two axes (ACL vs policy) by two states (new vs existing).
ACLs disabled	`aws_s3_bucket_ownership_controls` = `BucketOwnerEnforced`	AC-3, AC-6	AWS default since April 2023. An ACL-enabled bucket is itself a CIS and Security Hub finding.
Deny non-TLS	`aws_s3_bucket_policy` (`aws:SecureTransport`)	SC-8, SC-8(1)	**The gap the official labs never close anywhere.**
Deny wrong-key uploads	`aws_s3_bucket_policy` (`s3:x-amz-server-side-encryption`)	SC-28	Turns encryption from a default into an enforced condition.
Lifecycle rules	`aws_s3_bucket_lifecycle_configuration`	AU-11, SA-9	Retention is a control, and unbounded logs are an unbounded bill.

Property	Resource	Control	Why it is in the floor
Four required tags	provider `default_tags`	CM-6, CM-8	Boundary enumeration by API call instead of by spreadsheet.

Two properties are **not** in the floor, deliberately, and each lab that needs them says so: Object Lock (Lab 2.5, evidence only) and cross-account log delivery (documented, not built, see Lab 5.2).

What changed, and why

Read this if you have taken the official labs. Everything here is a correction rather than a preference.

1. A state backend now exists (new Lab 2.2). The official labs build no backend anywhere, yet Lab 4.3's pipeline ran `terraform init` on a fresh runner and the capstone required `Apply` on merge to `main`. With local state, that runner starts with empty state, so the first apply after a merge either collides with existing resources or duplicates them. The capstone's Layer 3 was not completable as written. Lab 2.2 fixes it before anything depends on it.

2. Lab 2.3's log bucket is now versioned. The official architecture prose says "both buckets enforce ... versioning" while its code versioned only the primary. The bucket holding audit records was less protected than the bucket holding data. Verified against their own resource count: 11 resources, no `aws_s3_bucket_versioning.log`.

3. TLS enforcement exists (SC-8). They never use `aws:SecureTransport` in any lab, and never cited SC-8. Encryption at rest was covered in Chapter 2; encryption in transit was covered nowhere. This is also a `tfsec` HIGH, meaning their own Chapter 4 gate would fail their own Chapter 2 artifact.

4. AWS KMS is taught before the capstone requires it. They teach CMEK only in Lab 2.4, on GCP. The capstone's Layer 1 required AWS CMKs with rotation. Students met `aws_kms_key` for the first time in a graded deliverable.

5. Log delivery uses a bucket policy, not an ACL. They set `BucketOwnerPreferred` and applied a `log-delivery-write` ACL, re-enabling ACLs on the log bucket to do it. That works, and it trips Security Hub's "S3 general purpose buckets should have ACLs disabled." These notes use the modern grant to `logging.s3.amazonaws.com` with `aws:SourceArn` and `aws:SourceAccount` confused-deputy conditions. The `depends_on` dance disappears with it.

6. AU-6 is no longer claimed by Lab 2.3. AU-6 is audit *review, analysis, and reporting*. Shipping logs into a bucket is AU-3 plus AU-11. They claim AU-6 in a lab where nothing reads the logs. These notes move the claim to Lab 5.2, which now stands up an Athena table over the trail and runs a query, because that is what AU-6 costs.

7. Versioning's control mapping is defended, not asserted. They map versioning to CM-6. These notes map it to CP-9 and SI-7, and on log buckets to AU-9, and explains why CM-6 was the weaker argument.

8. CloudTrail data events are on for the evidence vault (Lab 5.2). They say "we do not enable data events here," which left object-level access to the evidence vault unaudited. The vault is the one bucket where you must know who read what. Cost is called out explicitly.

Cost and time

This costs more than the official labs. That is the honest consequence of "production grade," and every lab states its own number. Running the whole curriculum and destroying same-day:

Lab	Marginal cost	Time
0.1	free (sets the budget cap)	90 min
2.2	under $0.01	20 min
2.3	~$1/mo per KMS key, prorated to cents	60-75 min
2.4	~$0.06 per key version per month	45-60 min
2.5	under $0.01	45 min
3.3, 3.4	free (plan only)	105 min
4.3, 4.4	free	150 min
5.2	$1-8 if left running a month	75 min
5.4	pennies	75-90 min
6.1	free	60-75 min

Destroy same-day and the whole curriculum lands under $5. Leave Security Hub and a few KMS keys running for a month and it is $10-15. KMS keys bill $1/month each whether you use them or not, and a scheduled key deletion still bills through its waiting period.

Conventions

- **Run every command inside the container**, if you set one up in Lab 0.1. That is where the toolchain and your cloud logins live. Running on your own machine means installing the ten tools and logging in a second time, which is a separate setup rather than a shortcut.
- `evidence/` **lives at the repository root**, not inside the workspace you happen to be in. Capture blocks anchor to it with `EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-X-Y"`, so they work from any directory.
- Every lab states cost and time before the first command.
- Every control citation appears as a comment at the exact line that enforces it.

- Every lab ends with cleanup that actually works, verified against versioned and Object-Locked buckets.
- Placeholders are `<your-sandbox>`, `<your-vault>`, `your-gcp-project`. Substitute yours.
- No lab asks you to run something it has not told you the cost of.